MacroMate

Complete Product & Technical Documentation

An AI-assisted nutritional concierge for Android -- macro-aware Swiggy meal discovery, manual/photo food logging, and daily/weekly tracking.

Generated September 20, 2026

Repository reference: macromate/ (com.aistudio.macromate.rshqy)

Platform: Android (Kotlin, Jetpack Compose, Room, Gradle)

Table of Contents

Table of Contents 2

Executive Summary 4

Product Overview 5

Feature Overview 6

Application Screens 8

- Screen: Sign Up (Authentication) . . . 9
- Screen: Onboarding -- Goal Selection (step 2 of 4) . . . 10
- Screen: Assistant (Chat) -- AI Macro Discovery Results . . . 11
- Screen: Basket (Cart) . . . 12
- Screen: Tracker -- Daily & Weekly Progress . . . 13
- Screen: Profile . . . 14
- Additional Screens & States . . . 15

Complete User Journey 17

- 1. First run: account creation -> onboarding -> main app . . . 17
- 2. Discovering and ordering a meal through chat . . . 17
- 3. Logging a meal that wasn't ordered through the app . . . 18
- 4. Linking a real Swiggy account . . . 19

System Architecture 20

Frontend Architecture 21

"Backend" Architecture 22

API Architecture 23

Database Architecture 24

AI/ML Architecture 25

Integrations 26

Data Flow 27

Repository / Code Structure 28

Authentication & Security 29

Deployment Architecture 30

Testing 31

Configuration 32
Known Limitations 33
Future Improvement Areas 34
Glossary 35
Quick Start for Developers 36

Executive Summary

MacroMate is a native Android application that helps a single user track daily nutrition (calories, protein, carbs, fat) and find food -- from Swiggy restaurants or from meals they cook themselves -- that fits their remaining macro budget for the day. It combines an on-device Room database for persistence, a scripted "ReAct-style" recommendation agent for scoring dishes against a user's targets, a real integration with Swiggy's public MCP (Model Context Protocol) server for live restaurant/menu data, and Google's Gemini API for natural-language search interpretation, response writing, and photo-based food recognition.

There is no backend server. This is a self-contained client app: everything that isn't rendered on-screen is either stored locally in a SQLite database (via Room) or fetched directly from two third-party HTTP APIs (Swiggy MCP, Gemini). There is one user role -- an authenticated end user -- and no administrative surface.

The product exists to solve a narrow, real problem: macro tracking apps typically stop at logging what you ate; MacroMate also tries to answer "given what I have left today, what should I eat next, and can I order it right now." The chat-based search is the product's centerpiece; the tracker, basket, and profile screens support that core loop.

Product Overview

Problem statement. People trying to hit daily macro targets (for fat loss, muscle gain, or maintenance) face two separate chores: figuring out what to eat that fits their remaining budget, and then logging it accurately afterward. Off-the-shelf calorie trackers handle logging but not recommendation; food delivery apps handle ordering but know nothing about the user's targets.

Solution. MacroMate merges both loops into one app. A user sets a goal (Cut / Maintain / Bulk) once during onboarding, and the app auto-derives daily calorie/protein/carb/fat targets from their weight and goal (a standard TDEE-based formula). From there, two logging paths exist side by side:

1. **Discover through chat** -- describe what you want to eat ("high protein lunch under Rs.300"), and the agent searches restaurants (real Swiggy data if the account is linked, otherwise a built-in offline demo catalog), filters by allergies/diet, scores the remaining dishes against the user's *remaining* macros for the day, and returns a ranked shortlist with reasoning. Accepted dishes go into a Basket; confirming the basket logs those macros for the day and hands off to the real Swiggy app via a deep link.
2. **Log directly** -- for meals not ordered through the app (home-cooked food, a restaurant visit), the user can type a plain-language description or take/upload a photo of the meal, and Gemini estimates the macros to pre-fill the log (still fully editable before saving).

Target users. Individuals already tracking fitness goals who order food delivery regularly -- the app assumes an Indian delivery context (Swiggy, Rs. pricing, Indian cuisine keywords in the demo catalog).

User roles. Single role: an authenticated app user. Each device holds its own local accounts (multiple can be registered on one device via the sign-up/sign-in screen), but nothing syncs across devices -- there is no server-side account system.

Core capabilities.

- Goal-based macro target calculation
- AI-assisted, macro-aware restaurant/dish discovery (mock or live Swiggy data)
- Cart -> simulated Swiggy order placement, with automatic macro logging
- Manual meal logging, either typed, described in natural language for AI estimation, or photographed for AI vision estimation
- Daily and 7-day macro tracking with progress visualizations
- Optional real Swiggy account linking via OAuth
- Optional Gemini API key for real AI features (falls back to deterministic behavior without one)

Major workflows (detailed below in Complete User Journey): account creation -> onboarding -> chat-based discovery -> basket -> simulated order; manual/photo logging; Swiggy account linking; daily/weekly tracking review.

Feature Overview

Feature	Description	Primary User	Importance
Sign up / Sign in	Local, on-device account creation and authentication by email + password	All users	Critical -- gates all other screens
4-step onboarding	Collects name, age, weight, height, goal (Cut/Maintain/Bulk), allergies/diet, cuisine preferences; auto-calculates daily macro targets	New users	Critical -- every downstream feature depends on the targets it sets
AI chat discovery	Natural-language meal search; agent interprets the query, searches restaurants, filters by diet/allergy, scores dishes against remaining daily macros, narrates results	All users	Core -- the product's centerpiece
Agentic re-search	If the first search returns too few good macro matches, the agent (when Gemini is configured) asks for a different search angle and retries once	All users	Secondary -- quality-of-result improvement, invisible unless it fires
Swipeable result cards	Dish cards with macro breakdown, AI reasoning ("Why this?"), swipe-to-dismiss, add-to-cart	All users	Core -- the primary result-consumption UI
Basket / Cart	Quantity controls, aggregate macro/price totals, "Confirm & Order" that logs macros and deep-links to the real Swiggy app	All users	Core -- completes the discovery loop
Manual meal logging	Typed entry of a meal's name/macros/price for anything not ordered through the app	All users	Core -- needed for home-cooked/non-Swiggy meals
AI text-to-macro estimation	Free-text food description -> Gemini-estimated calories/protein/carbs/fat, pre-filling the manual log form	All users	High -- removes the need to already know a meal's macros
AI photo-to-macro estimation	Camera photo or gallery upload of a meal -> Gemini vision identifies the food and estimates macros	All users	High -- same as above, without needing to describe the food in words

Feature	Description	Primary User	Importance
Daily macro tracker	Animated progress rings/bars for calories, protein, carbs, fat against daily targets; list of today's logged meals	All users	Core -- the tracking half of the product
7-day history chart	Bar chart of daily calorie totals for the past week	All users	Secondary -- trend visibility
Swiggy account linking	Real OAuth 2.1+PKCE connection to Swiggy's MCP server, switching the app from offline demo data to live restaurant/menu search	Users who want real ordering	High -- without it the app is a fully-functional demo on mock data
Order history	Read-only list of past simulated orders with item breakdown and macro totals	All users	Secondary
AI engine selector (Gemini/Claude)	UI toggle in Profile for choosing an AI provider	All users	Cosmetic only -- the Claude option is not wired to any backend; only Gemini is implemented
Reset all data	Clears all logs and order history for the current user	All users	Secondary -- utility/debug feature

Application Screens

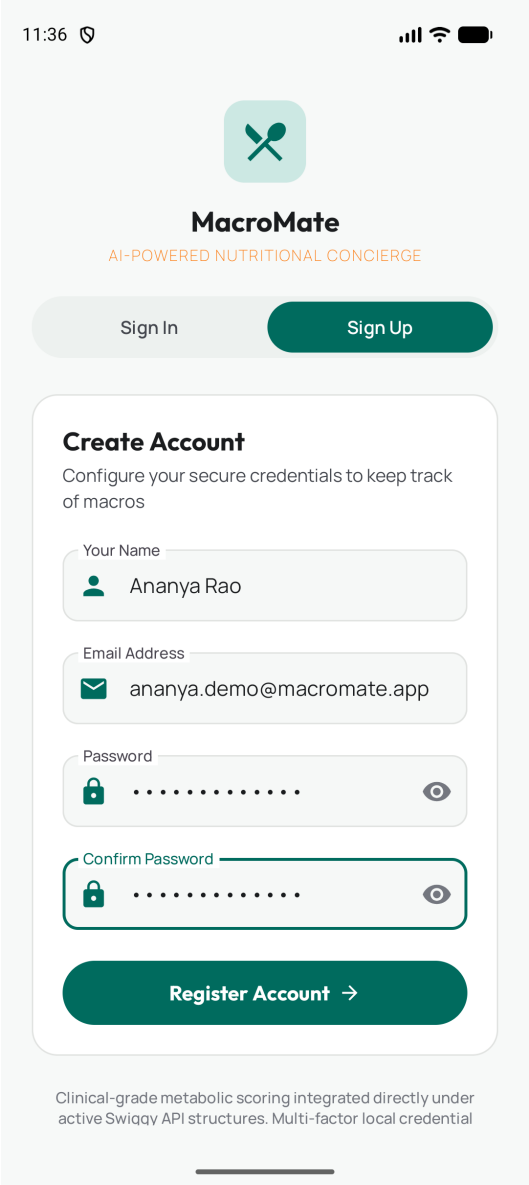
Screens below were captured from a running debug build on an Android emulator, populated with realistic demo data (a fictional account, "Ananya Rao", created for this documentation run) -- not real user data.

Screen: Sign Up (Authentication)

Purpose: Creates a new local account (email + password) so the app can persist a profile and macro logs for that user.

User actions: Enter name, email, password, confirm password; tap "Register Account". Validation covers blank fields, email format, and a 6-character minimum password.

Backend interaction: MacroMateViewModel.signUp() writes a UserAccount row and a blank UserProfile (weightKg = 0.0, the sentinel that routes to Onboarding next) via Room -- entirely on-device, no network call.

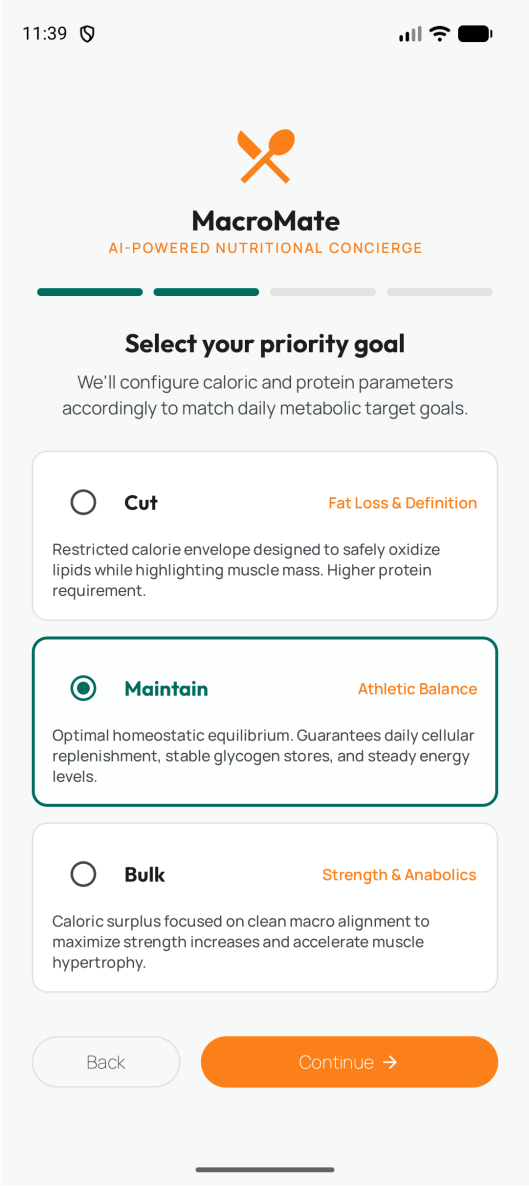


Screen: Onboarding -- Goal Selection (step 2 of 4)

Purpose: Captures the user's priority (Cut / Maintain / Bulk), which drives the TDEE-based daily macro target calculation.

User actions: Select one of three goal cards, each with a short rationale, then Continue.

Backend interaction: No backend call here -- the selection is held in Compose state until the final step, when calculateMacrosForGoal() derives dailyCalories/Protein/Carbs/Fat and writes the completed UserProfile.

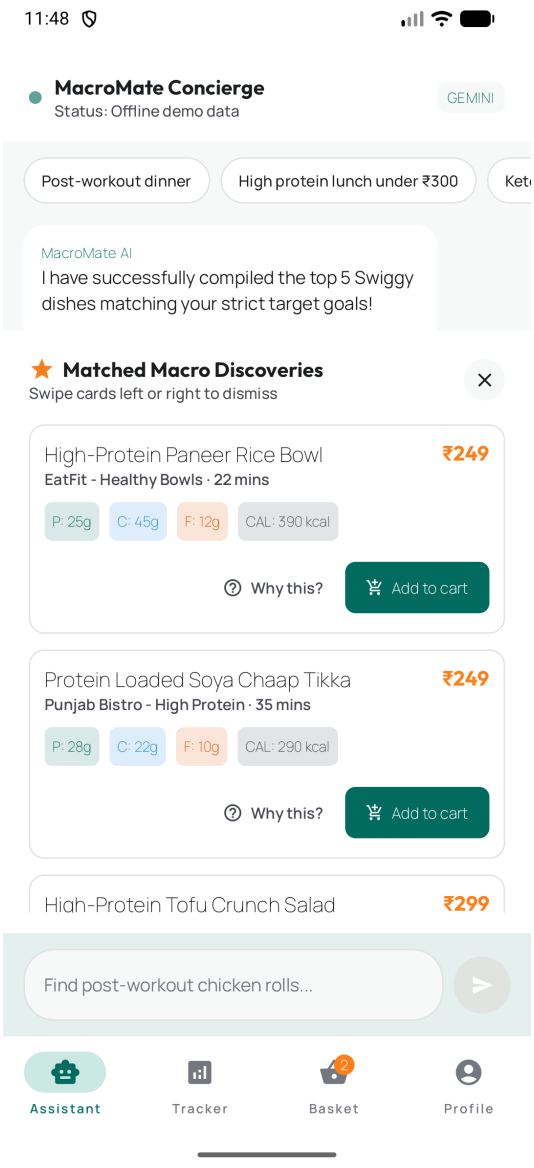


Screen: Assistant (Chat) -- AI Macro Discovery Results

Purpose: The product's centerpiece: natural-language meal search, ranked against the user's remaining daily macros, with dietary filtering already applied.

User actions: Type a request or tap a quick prompt; tap "Why this?" for the reasoning behind a pick; tap "Add to cart" to stage a dish for ordering.

Backend interaction: MacroMateViewModel.sendMessage() -> MacroMateAgent.searchAndRankMeals() -> SwiggyClient (mock catalog here, since no Swiggy account is linked in this run) -> deterministic scoring/filtering -> results rendered as swipeable cards. Both results shown are vegetarian, correctly honoring the dietary restriction set during onboarding.

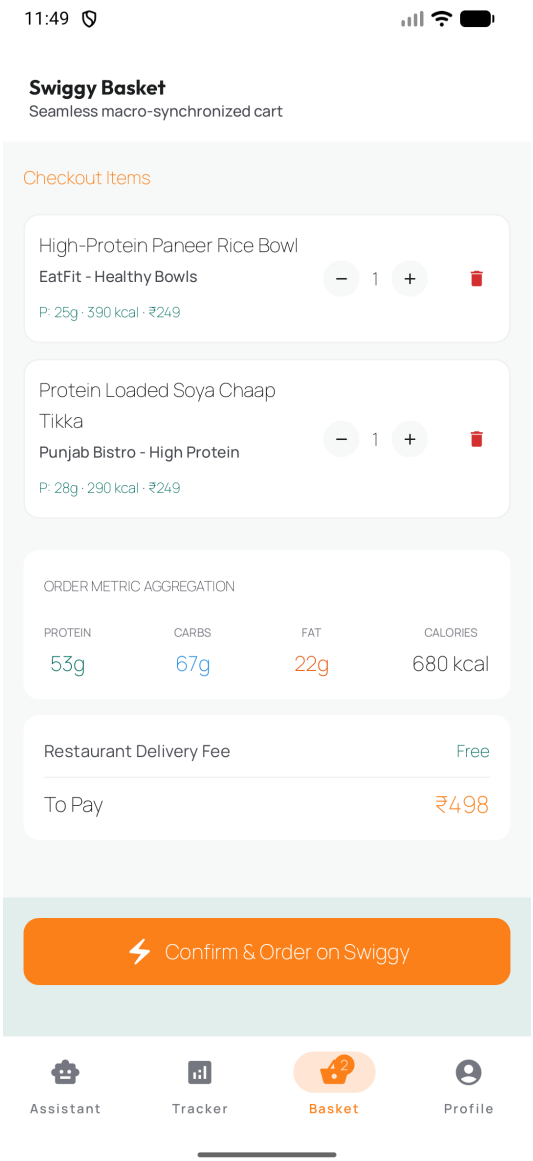


Screen: Basket (Cart)

Purpose: Aggregates the dishes added from chat, with running macro and price totals, before handing off to Swiggy for the actual order.

User actions: Adjust quantities with +/-, remove items, or tap "Confirm & Order on Swiggy".

Backend interaction: Confirming calls confirmCartAndPlaceOrder(): writes an OrderHistory row, logs each item as a MacroLog for today, clears the cart, and fires a swiggy:// deep link (falling back to the Swiggy website).

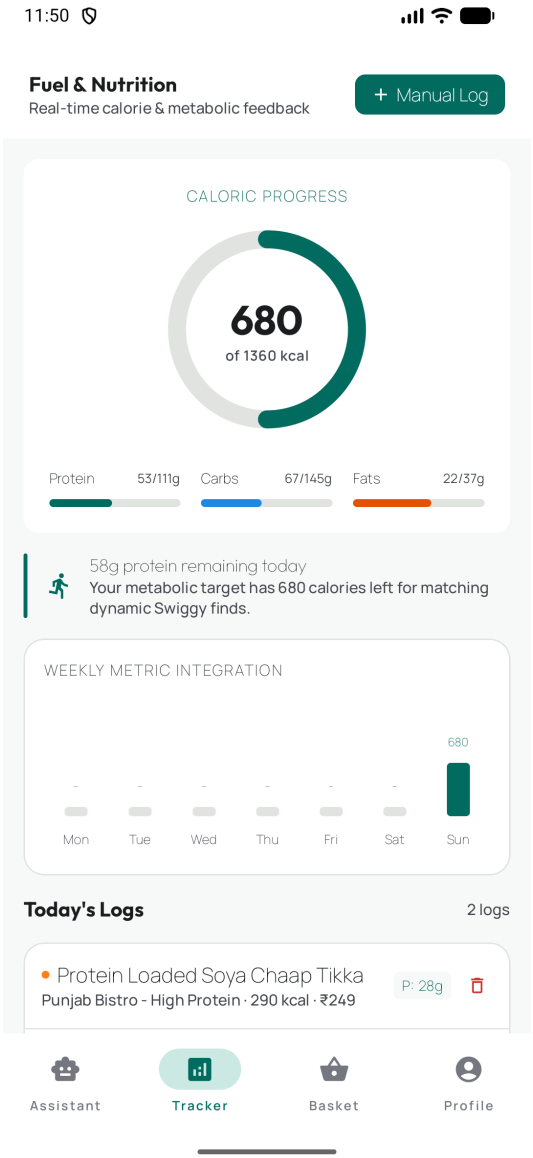


Screen: Tracker -- Daily & Weekly Progress

Purpose: Shows today's macro progress against targets and a 7-day calorie history, plus the list of everything logged today.

User actions: Review the calorie ring and per-macro bars; scroll the weekly chart; tap "Manual Log" to add an untracked meal; swipe/tap to delete a logged entry.

Backend interaction: Reads live from Room via StateFlow (todayLogs, allLogs) -- the ring/bars/chart recompute automatically as new MacroLog rows are written elsewhere in the app (e.g. right after the order shown on the previous page).

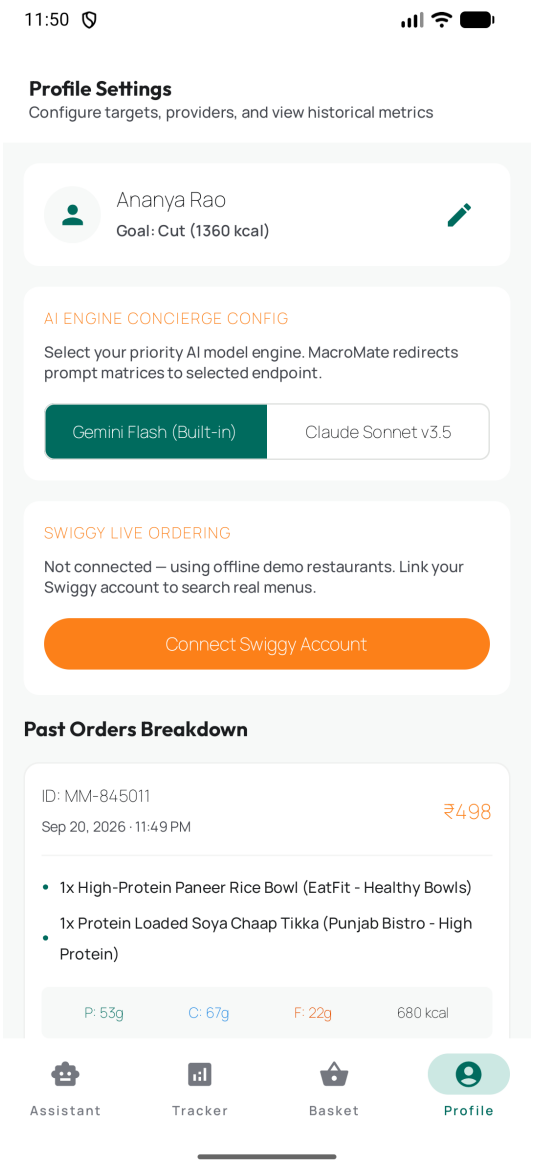


Screen: Profile

Purpose: Account and configuration hub: identity, AI engine preference, Swiggy account linking, and order history.

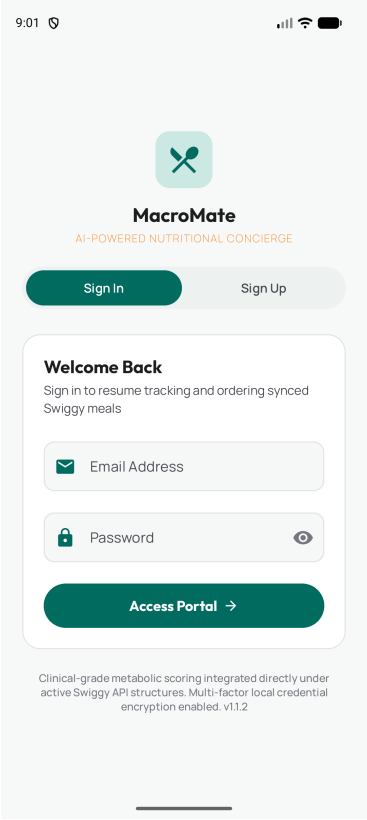
User actions: Edit profile fields, switch the AI engine toggle (Gemini is the only one actually wired), tap "Connect Swiggy Account" to start the OAuth flow, review past simulated orders.

Backend interaction: The order shown here is the exact one just placed from the Basket screen -- note the matching Rs.498 total and 53g/67g/22g/680kcal breakdown, confirming the order-to-log pipeline end to end.

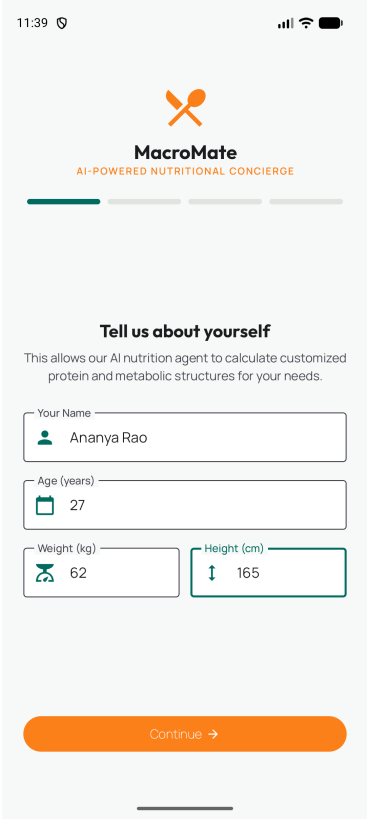


Additional Screens & States

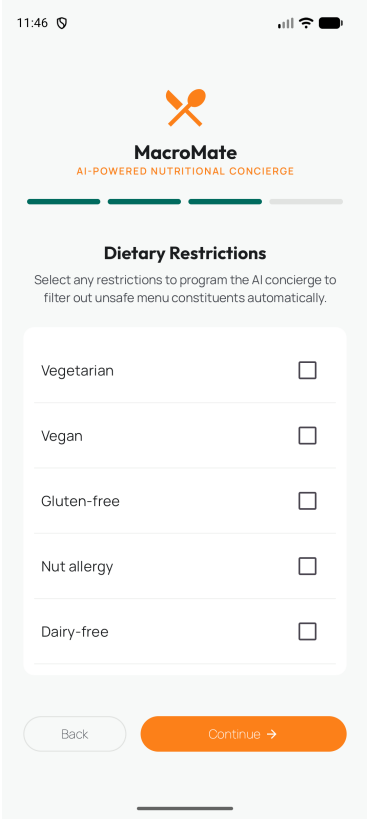
Smaller reference captures of the remaining screens and states, for completeness.



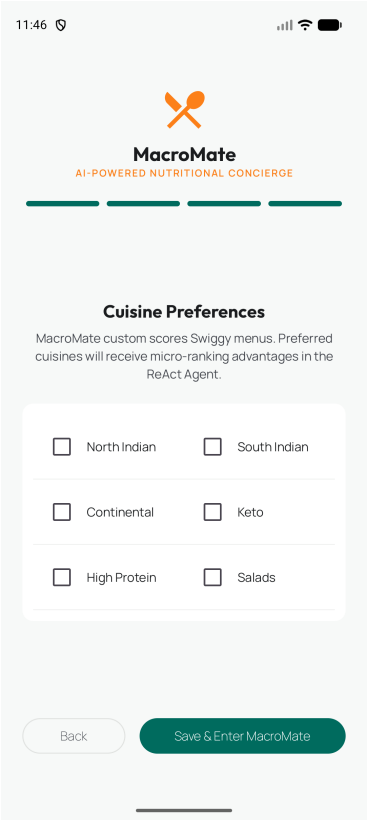
Sign In tab



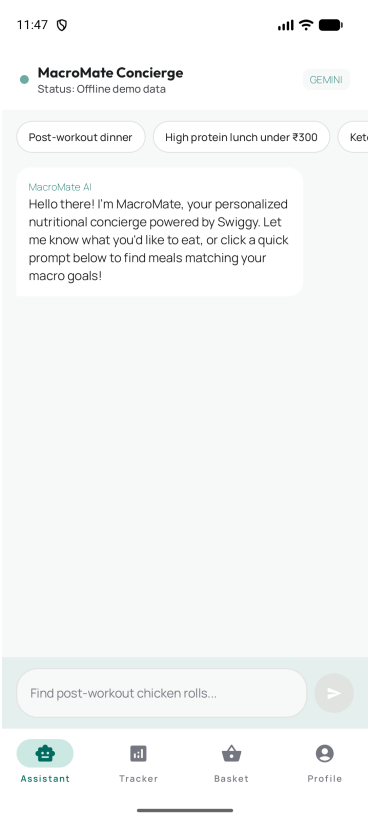
Onboarding step 1: bio (name/age/weight/height)



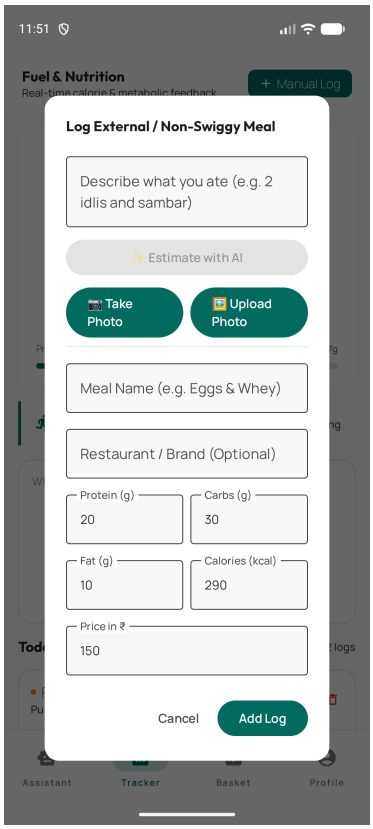
Onboarding step 3: dietary restrictions



Onboarding step 4: cuisine preferences



Assistant screen, empty/welcome state



Manual Log dialog: AI text/photo estimation

Complete User Journey

1. First run: account creation -> onboarding -> main app

```
Launch app
  |
  v
AuthScreen (Sign Up tab)
  |  name, email, password, confirm password
  |  MacroMateViewModel.signUp() -> validates -> writes UserAccount
  |  + a blank UserProfile (weightKg = 0.0, a sentinel meaning
  |    "onboarding incomplete") to Room
  v
MainActivity observes userProfile.weightKg <= 10.0 -> routes to OnboardingScreen
  |
  v
OnboardingScreen (4 steps)
  1. Name / Age / Weight / Height
  2. Goal: Cut / Maintain / Bulk
  3. Dietary restrictions (vegetarian, vegan, gluten-free, nut allergy, dairy-free)
  4. Cuisine preferences (North Indian, South Indian, Continental, Keto, High
     Protein, Salads)
  |  "Save & Enter MacroMate" -> calculateMacrosForGoal() derives daily
  |  calories/protein/carbs/fat from weight+goal, writes the completed
  |  UserProfile
  v
MainActivity re-observes profile -> weightKg now > 10 -> routes to the 4-tab
main workspace (Assistant / Tracker / Basket / Profile), landing on Assistant
```

2. Discovering and ordering a meal through chat

```
ChatScreen: user types or taps a quick prompt (e.g. "high protein lunch
under Rs.300")
  |
  v
MacroMateViewModel.sendMessage() -> computes remaining daily macros
(daily target - today's logged total) -> MacroMateRepository.runAgent()
  |
  v
MacroMateAgent.searchAndRankMeals() [the ReAct loop -- see AI/ML Architecture]
  1. (if Gemini configured) interpret the raw query into a concrete search
     keyword
  2. SwiggyClient.searchRestaurants(keyword) -- live Swiggy MCP if the
     account is linked, else the offline demo catalog
  3. For up to 3 restaurants: fetch menu, filter out dishes that violate
     allergies/diet, score remaining dishes:
     score = 0.5-proteinMatch + 0.3-calorieMatch + 0.2-priceMatch
     (each match term is 1 - |estimate - target| / target, clamped to
     [0,1]; +0.05 if the restaurant's cuisine matches a stated preference)
  4. (if Gemini configured, and fewer than 2 dishes score above 0.4) ask for
     a different search angle and retry once, merging results
  5. Sort, take top 5
  6. (if Gemini configured) one batched call drafts the conversational
     reply and a specific reasoning sentence per dish; otherwise a
     template-based reasoning string is used
  v
ChatScreen renders: chat bubble with the summary + a swipeable "Matched
Macro Discoveries" card stack (macro pills, "Why this?" dialog, "Add to
cart")
  |  user taps "Add to cart" on one or more dishes
  v
CartScreen: quantity controls, aggregate macro/price totals
  |  "Confirm & Order on Swiggy"
  v
MacroMateViewModel.confirmCartAndPlaceOrder():
  - writes an OrderHistory row (simulated order + swiggy IDs)
  - writes one MacroLog row per cart item for today
  - clears the cart
  - posts a system chat message confirming the "order"
  - fires an Intent(ACTION_VIEW, "swiggy://cart") deep link, falling back to
    the Swiggy website if the Swiggy app isn't installed
  v
TrackerScreen: today's progress rings/bars and logged-meal list now reflect
the new totals
```

3. Logging a meal that wasn't ordered through the app

```
TrackerScreen -> "Manual Log" -> dialog opens
  |
  +- Path A: type a description ("2 idlis and sambar") -> "Estimate with AI"
  |     -> MacroMateViewModel.estimateMacrosFromDescription() -> Gemini
  |       generateJson() -> pre-fills calories/protein/carbs/fat (editable)
  |
  +- Path B: "Take Photo" -> runtime CAMERA permission request -> system
  |     camera (ActivityResultContracts.TakePicturePreview) -> bitmap ->
  |     estimateMacrosFromImage() -> Gemini vision (generateJsonFromImage,
  |       base64 inline image + prompt) identifies the food and estimates
  |       macros, also suggesting a meal name
  |
  +- Path C: "Upload Photo" -> system photo picker
  |     (ActivityResultContracts.GetContent) -> same estimation path as B
  |
  +- Path D (always available): type the meal name and macros directly
  |
  v
On any path (including AI failure -- no key, network error, bad parse -- which
falls back cleanly to "enter macros manually" with the numeric fields still
editable): "Add Log" -> MacroMateViewModel.logManualMeal() writes a MacroLog
row with source = "manual", distinguishing it in the tracker from
Swiggy-sourced entries.
```

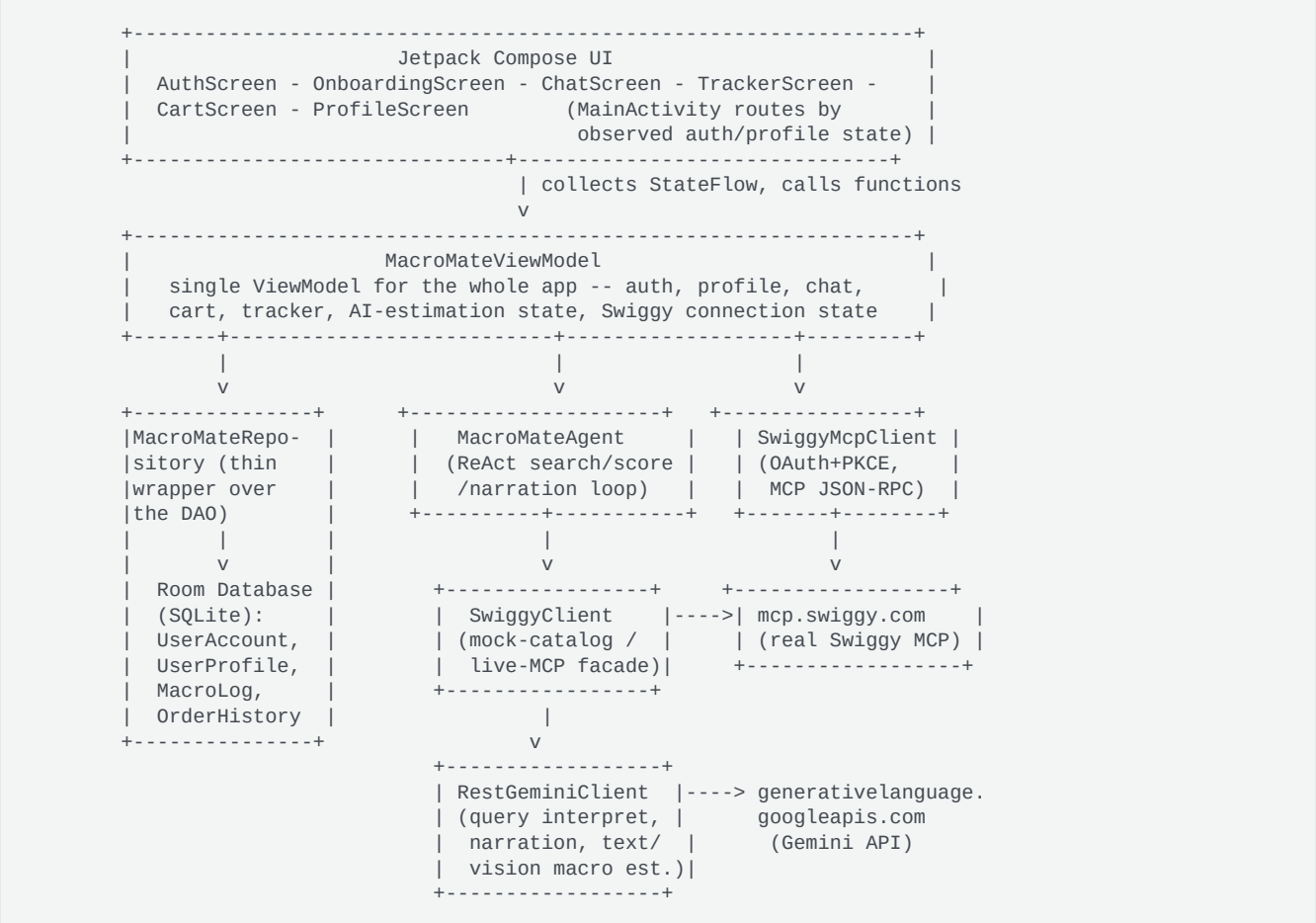
4. Linking a real Swiggy account

```
ProfileScreen -> "Connect Swiggy Account"
  |
  v
SwiggyMcpClient.connect():
  1. Dynamic client registration against mcp.swiggy.com/auth/register
     (no pre-issued credentials needed)
  2. Builds an OAuth 2.1 + PKCE authorization URL, opens it in the system
     browser (Intent(ACTION_VIEW))
  3. User completes Swiggy's phone-number + OTP login in the browser
  4. Swiggy redirects to http://localhost:8971/callback -- caught by a
     loopback ServerSocket the app opens locally (the RFC 8252 native-app
     pattern), since Swiggy only allowlists https/localhost redirect URIs
  5. Exchanges the authorization code (+ PKCE verifier) for an access token
     at mcp.swiggy.com/auth/token
  6. Calls the get_addresses MCP tool and stores the first saved address --
     every subsequent Food tool call needs one
  v
ProfileScreen shows "Connected"; ChatScreen's status line switches from
"Offline demo data" to "Live Swiggy data"; SwiggyClient now routes
searchRestaurants/getMenu to real MCP calls, falling back silently to the
offline catalog if a live call fails.

Swiggy's v1.0 API issues no refresh token -- the access token lasts ~5 days,
after which the user must repeat this flow.
```

System Architecture

Single-module Android app, no separate backend. Everything below runs on-device except the two labelled external API calls.



Everything left of the two external-API boxes runs entirely on the device; nothing here requires or has a server component of MacroMate's own.

Frontend Architecture

Framework. Jetpack Compose (Material3) throughout -- there is no legacy XML view layer. `MainActivity` is the single Activity; there is no navigation-library graph. Screen routing is driven purely by observed state:

```
if (currentUserId == null) AuthScreen(...)
else if (profile == null) LoadingSpinner
else if (profile.weightKg <= 10.0) OnboardingScreen(...)   // sentinel for "incomplete"
else MainWorkspaceScaffold(...)  // 4-tab Scaffold: Assistant/Tracker/Basket/Profile
```

State management. One `MacroMateViewModel` (an `AndroidViewModel`) backs every screen. All cross-cutting state (auth, profile, chat messages, cart, tracker logs, AI-estimation loading flags, Swiggy connection status) is exposed as `StateFlow`, collected in Compose via `collectAsState()`. Room queries that return `Flow` are chained through `flatMapLatest` keyed on the current user id, so switching (or logging out of) accounts automatically re-subscribes every downstream flow to the new user's data.

Theming. A single light Material3 theme ("Neon Emerald & Slate"): teal (`NeonEmerald`) as the primary/status/AI-branding accent, Swiggy's brand orange (`OrangeSwiggy`) reserved for genuinely Swiggy-commerce actions (pricing, "Confirm & Order," live-Swiggy indicators), off-white/slate neutrals for backgrounds and surfaces. Typography uses two bundled Google variable fonts -- **Outfit** for headlines/display and **Manrope** for body/UI text -- selected via `FontVariation.Settings` (graceful fallback to a font's default weight on API < 26). A shared `tactilePress()` Compose modifier (spring-based scale-down-on-press) standardizes physical button feedback app-wide; it formalizes a spring-press pattern that had already been hand-rolled once, independently, in `CartScreen`'s quantity buttons.

Notable UI patterns.

- Swipe-to-dismiss result cards (`draggable` + threshold-based dismissal) in Chat
- Canvas-drawn progress arcs and an animated bar chart in Tracker
- A deliberate move away from "every section is its own elevated `Card`" -- Chat and Tracker (redesigned this session) use hairline dividers and plain grouped containers for repeated list content, reserving real elevation for single hero widgets (e.g. the calorie-progress ring)
- Runtime permission handling for the camera path, and the system photo picker (`ActivityResultContracts.GetContent`, no permission required) for gallery uploads

"Backend" Architecture

MacroMate has no backend server, no REST/GraphQL API of its own, and no cloud database. This section documents what plays that role instead:

- **On-device persistence layer** -- `MacroMateRepository` is a thin suspend-function wrapper over a Room `DatabaseDao`, which is the closest thing to a "service layer" in this app. It has no business logic beyond delegating to the DAO and to `MacroMateAgent`.
- **External API #1: Swiggy MCP** (`SwiggyMcpClient`) -- a real third-party server MacroMate authenticates against and calls over HTTP/JSON-RPC. Not something MacroMate operates or controls.
- **External API #2: Gemini** (`RestGeminiClient`) -- a real third-party REST API called directly over HTTPS/JSON. Also not operated by MacroMate.

There is no session/auth server: "login" is a local email+password check against the on-device `UserAccount` table, and the "logged in" state is a plain string (the user's email) persisted in `SharedPreferences`.

API Architecture

MacroMate does not expose any API -- it is a consumer of two external ones.

Swiggy MCP (Model Context Protocol, streamable HTTP / JSON-RPC 2.0, `https://mcp.swiggy.com/food`):

Tool called	Purpose	Called from
`get_addresses`	Fetch the user's saved delivery addresses; the first one is cached and reused for every subsequent Food call	Once, right after OAuth completes
`search_restaurants`	Restaurant search by query + address	`SwiggyClient.searchRestaurants()` when live
`get_restaurant_menu`	Menu for a given restaurant + address	`SwiggyClient.getMenu()` when live

Auth for these calls: OAuth 2.1 + PKCE, dynamic client registration (RFC 7591), bearer token on every MCP call, `Mcp-Session-Id` tracked from the `initialize` handshake. Response payload field names are not fully pinned in Swiggy's public docs, so the client parses defensively (a small breadth-first search for the first array of restaurant/dish-shaped objects) rather than assuming an exact schema.

Gemini API (`https://generativelanguage.googleapis.com/v1beta/models/gemini-3.5-flash:generateContent`):

Call	Purpose	Called from
`generateText`	Free-text generation (has a local offline-simulation fallback baked in, independent of the newer JSON-based paths)	Legacy path; not used by the current chat flow
`generateJson`	Structured JSON completion (`responseMimeType: application/json`) -- used for query interpretation, the agentic-retry suggestion, final chat narration, and text-based macro estimation	`MacroMateAgent`, `MacroMateViewModel.estimateMacrosFromDescription`
`generateJsonFromImage`	Same as above, with an `inline_data` (base64 JPEG) part prepended to the prompt -- Gemini vision	`MacroMateViewModel.estimateMacrosFromImage`

Auth: a single API key (`BuildConfig.GEMINI_API_KEY`) appended as a query parameter. Every Gemini call resolves to `null`/a safe fallback on missing key, HTTP failure, or unparsable response -- nothing in the app throws or crashes when Gemini is unavailable.

Database Architecture

Room (SQLite), database name `macromate_database`, schema version 2, `fallbackToDestructiveMigration()` (any future schema bump wipes and recreates rather than migrating -- acceptable for a local-only app with no irreplaceable server-backed data, but real data loss on a real upgrade if this project ever ships past debug builds).

```
UserAccount                      UserProfile
---------------------            ---------------------
email (PK)                       userId (PK, = email; "default_user" if unset)
passwordHash  <- misnamed: plaintext   name
name                              age, weightKg, heightCm
hasCompletedOnboarding            goal (Cut/Maintain/Bulk)
createdAt                         dailyCalories/ProteinG/CarbsG/FatG
                                   allergiesJson, cuisinePrefsJson (JSON arrays)
                                   llmProvider ("Gemini"/"Claude" -- cosmetic)
                                   createdAt

MacroLog                          OrderHistory
---------------------            ---------------------
id (PK, autogen)                 orderId (PK)
userId                            userId
dateString ("YYYY-MM-DD")         itemsJson (JSON array of item descriptions)
name, restaurant                  totalProtein/Carbs/Fat/Calories
proteinG/carbsG/fatG/calories      totalPrice
price                              swiggyOrderId (simulated)
source ("swiggy" | "manual")      createdAt
timestamp
```

No foreign-key constraints are declared; `userId` is a plain string column joining logically (not via Room relations) to `UserAccount.email`. All four tables are queried by `userId`, so every screen's data is implicitly scoped to the signed-in account. `MacroLog.dateString` (rather than deriving from `timestamp`) is what "today's logs" and the 7-day chart key off of.

AI/ML Architecture

MacroMate has two distinct, intentionally separated AI touchpoints. The **dish-ranking math is not AI** -- it's a fixed, tested formula. AI is used only to decide *what to search for* and *how to describe results/estimate macros for untracked food*.

1. Chat search -- agentic layer wrapping a deterministic core

```
user query
    |
    v (Gemini configured?)
 [yes] Gemini: interpret query -> concrete search keyword
 [no ] use the raw query text as-is
    |
    v
------------ deterministic core (no AI below this line) ------------
SwiggyClient.searchRestaurants(keyword) -> top 3 restaurants
  -> getMenu() each -> filter allergens/vegetarian/vegan (keyword rules)
  -> score = 0.5-proteinMatch + 0.3-calorieMatch + 0.2-priceMatch
    (+0.05 cuisine-preference boost), targets = remaining-daily-macros/3
-----------------------------------------------------------------
    |
    v (fewer than 2 dishes scored > 0.4, AND Gemini configured?)
 [yes] Gemini: suggest a different search angle -> repeat the deterministic
       core once more with the new keyword, merge results
 [no ] proceed with what was found
    |
    v (Gemini configured?)
 [yes] Gemini: one batched call writes the conversational reply + one
       reasoning sentence per dish, given the already-ranked list and scores
 [no ] template-based reasoning strings, fixed response copy
    |
    v
ChatScreen renders results
```

Without a Gemini key, this entire flow still works -- every "if Gemini configured" branch has a deterministic, tested fallback. The scoring formula above is covered by `MacroMateAgentScoringTest` and cannot be altered by anything the model says.

2. Macro estimation for manual logging

Two entry points, same output shape: `{calories, proteinG, carbsG, fatG}` (plus an optional suggested meal name for the photo path).

- **Text**: a description string -> one `generateJson` call with a nutritionist-framed prompt.
- **Photo**: a bitmap (camera capture, compressed to JPEG ~80% quality, or a gallery-picked image decoded via `MediaStore.Images.Media.getBitmap`) -> base64-encoded and sent as Gemini vision input via `generateJsonFromImage`, asking the model to both identify the food and estimate its macros.

Both paths fail closed: no API key, a network error, or an unparsable response all resolve to `null`, and the calling UI shows "Couldn't estimate -- enter macros manually below" with the numeric fields left fully editable -- never a crash, never a silently wrong number presented as fact.

Integrations

Integration	Purpose	Sync/Async	Failure handling
Swiggy MCP (`mcp.swiggy.com`)	Real restaurant/menu search and the OAuth account link	Async (suspend/coroutine HTTP calls)	Any live-call exception logs a warning and `SwiggyClient` falls back to the offline demo catalog; a 401 clears the stored session and asks the user to reconnect
Google Gemini API (`generativelanguage.googleapis.com`)	Query interpretation, response narration, text/photo macro estimation	Async	Every call site treats a `null`/failed result as an expected, handled case -- the app has parallel deterministic behavior for "no AI" throughout, not just a loading spinner that hangs
Android system Camera / Photo Picker	Capturing or selecting a food photo	Async (Activity Result contracts)	Denied camera permission or a cancelled picker both resolve to no bitmap, no crash
Android implicit Intent (`swiggy://cart`, fallback `https://www.swiggy.com/`)	Hands off a "confirmed" cart to the real Swiggy app or website	Fire-and-forget	Falls back to the web URL if no app can handle the deep link

Data Flow

The end-to-end flows for every major action are diagrammed in-line where they're most useful rather than repeated here: the account-creation and chat-discovery-to-order flows are in **Complete User Journey** above, and the two AI/ML pipelines (chat search interpretation, and macro estimation for manual logging) are diagrammed in **AI/ML Architecture**. In every case, the flow is: Compose UI event -> MacroMateViewModel function -> Repository/Agent/Client -> Room DB and/or an external API (Swiggy MCP or Gemini) -> StateFlow update -> Compose recomposition.

Repository / Code Structure

Single Gradle module, `com.example` namespace (`applicationId com.aistudio.macromate.rshqy`), all Kotlin source under `app/src/main/java/com/example/`:

```
com.example/
+-- MainActivity.kt             -- single Activity; state-driven screen routing
+-- ui/
|   +-- screens/                -- one file per screen (Auth, Onboarding, Chat,
|   |                              Tracker, Cart, Profile) + their sub-composables
|   +-- viewmodel/
|   |   +-- MacroMateViewModel.kt  -- the app's single ViewModel
|   +-- theme/
|       +-- Color.kt, Type.kt, Theme.kt  -- Material3 theme, fonts
|       +-- Motion.kt             -- shared tactilePress()/spring-motion helpers
+-- data/
    +-- model/Models.kt           -- every Room entity + plain UI/network data class
    +-- local/
    |   +-- MacroMateDatabase.kt
    |   +-- DatabaseDao.kt
    +-- repository/
        +-- MacroMateRepository.kt  -- thin DAO/agent wrapper
        +-- MacroMateAgent.kt       -- the ReAct search/score/narrate loop
        +-- SwiggyClient.kt         -- mock-catalog / live-MCP facade
        +-- SwiggyMcpClient.kt      -- OAuth+PKCE, MCP JSON-RPC
        +-- RestGeminiClient.kt     -- Gemini REST calls
```

Tests live under `app/src/test/java/com/example/` (JVM/Robolectric unit tests) and `app/src/androidTest/java/com/example/` (one template instrumented test, unused by this project's actual test strategy). Bundled font resources live under `app/src/main/res/font/`.

Authentication & Security

Authentication model. Email + password, checked locally against the on-device `UserAccount` Room table. "Logged in" is just the user's email stored in `SharedPreferences` (`macromate_prefs`); there is no token, no session expiry, no server round-trip.

Known issue -- plaintext passwords. `UserAccount.passwordHash` is misleadingly named: `signUp` stores the raw password string as-is, and `signIn` compares with plain string equality. There is no hashing (bcrypt/Argon2/scrypt or otherwise). This is a real security gap, not a stylistic nit -- anyone with access to the device's app data (or a database backup, since `android:allowBackup="true"` is set) can read every stored password in cleartext. This is flagged explicitly as a defect to fix, not documented as if it were acceptable behavior.

Swiggy account linking. Uses real OAuth 2.1 with PKCE (RFC 7636) and dynamic client registration -- a materially stronger auth flow than the app's own login, because it's talking to a real third-party identity provider rather than a local table. The redirect is caught on a `127.0.0.1` loopback socket bound to a fixed local port, matching the native-app OAuth pattern Swiggy's docs require (HTTPS/localhost-only redirect URIs). Access tokens are stored in a separate `SharedPreferences` file (`swiggy_mcp`) and are not encrypted at rest.

Other notes. No rate limiting, no CORS concerns (no server), no input sanitization beyond basic blank/format checks on the auth form (email regex via `android.util.Patterns`, password length >= 6). The `GEMINI_API_KEY` build secret is the one credential genuinely worth protecting, and -- as found and fixed during this project's own review -- an earlier version of `.env.example` shipped a live, working key in the repository; it has since been replaced with a placeholder, but the exposed key should be treated as compromised and rotated.

Deployment Architecture

There is no deployment in the server/cloud sense -- this is a local Android debug build with no CI/CD pipeline, no Docker configuration, and no Play Store listing referenced anywhere in the project. What does exist:

- Gradle build types: `debug` (default, used throughout development and testing) and `release` (minification disabled, a `signingConfig` that reads `KEYSTORE_PATH`/`STORE_PASSWORD`/`KEY_PASSWORD` from environment variables or a local `my-upload-key.jks`, for producing a signed APK manually if ever needed).
- The Secrets Gradle Plugin injects `GEMINI_API_KEY` from a local `.env` file into `BuildConfig` at compile time; `.env` is not committed, only `.env.example` (a placeholder) is.
- Installation during development is via `adb install` to an emulator (AVD) or a physical device over USB or ADB-over-Wi-Fi -- there is no automated distribution channel.

Testing

Test framework: JUnit4 + Robolectric (JVM-executed Android unit tests; no real device/emulator required to run them) + `kotlinx-coroutines-test`. Roborazzi is configured as a dependency for screenshot testing but no baseline screenshots exist in this project.

Test file	Covers
`MacroCalculationTest`	The TDEE-based macro-target formula (Cut/Bulk/Maintain math, the 1200-calorie floor on aggressive cuts) -- tested against a duplicated copy of the formula, not the real `MacroMateViewModel.calculateMacrosForGoal` function directly
`CartViewModelTest`	Cart behavior via a real `MacroMateViewModel` under Robolectric: adding the same dish twice merges quantity without mutating a previously-observed `CartItem` reference; different dishes stay separate entries; reducing quantity to zero removes the item
`MacroMateAgentScoringTest`	The per-meal scoring target function (`mealTarget` -- daily/3, capped by what's actually left) and a regression check that a high-protein dish scores well on a fresh day (guards against the fixed per-meal-target bug found and corrected this session)
`SwiggyMcpParsingTest`	Defensive JSON/SSE parsing helpers in `SwiggyMcpClient` -- extracting a JSON-RPC response from an SSE stream vs. plain JSON, mapping nested restaurant/menu payloads, and paise-to-rupee price normalization
`ExampleRobolectricTest`, `ExampleUnitTest`, `GreetingScreenshotTest`	Template-generated tests; the Robolectric one now asserts the real app name after being fixed to match ("MacroMate", not the scaffold's "My Application"); the screenshot template test was removed because it referenced a composable that no longer exists in this project

What is not tested: there are no instrumented UI tests (`androidTest`) beyond an unused template, no end-to-end tests, and no automated tests for `RestGeminiClient`/`SwiggyMcpClient`'s actual network calls (they require a live key/session and are not mocked). All verification of the live Swiggy OAuth flow, the camera/photo estimation flow, and the visual UI redesign in this project was done as manual, screenshot-driven testing on an Android emulator (and briefly a physical device) during development -- not as part of the automated test suite. `./gradlew :app:testDebugUnitTest` runs the full unit suite; all listed tests pass as of the most recent run in this project's history.

Configuration

Variable	Where it's read	Purpose
`GEMINI_API_KEY`	`.env` (git-ignored), injected via the Secrets Gradle Plugin into `BuildConfig.GEMINI_API_KEY`	Enables real Gemini calls; absent or left as the `.env.example` placeholder -> the app runs fully on deterministic/offline fallbacks. Value: `GEMINI_API_KEY=<secret>`
`KEYSTORE_PATH`, `STORE_PASSWORD`, `KEY_PASSWORD`	Shell environment, read in `app/build.gradle.kts`'s release `signingConfig`	Only needed to produce a signed release APK; unused for normal debug development

No other runtime configuration exists -- no server URLs to point at different environments, no feature flags.

Known Limitations

- **Plaintext password storage** (see Authentication & Security) -- a real defect, not a stylistic choice.
- **Offline-first by default**: without a linked Swiggy account, all restaurant/menu data is a fixed, hardcoded 4-restaurant / 20-dish catalog -- realistic-looking but entirely static.
- **Swiggy session expiry with no refresh**: Swiggy's MCP v1.0 issues no refresh token; the access token lasts roughly 5 days, after which the user must repeat the full OAuth flow.
- **No cross-device sync**: accounts, logs, and cart state are local to one device's Room database; reinstalling the app or switching devices loses everything.
- `fallbackToDestructiveMigration()`: any future Room schema version bump wipes all local data rather than migrating it.
- `MediaStore.Images.Media.getBitmap` **is deprecated** (still functional at this project's `minSdk 24`, but Google recommends `ImageDecoder` on API 28+ -- not adopted here to avoid an API-level branch for a single, still-working call).
- **No automated instrumented/E2E test coverage** -- all UI-level correctness this session was verified manually, not by a repeatable test suite.
- **The Claude entry in the AI-engine selector does nothing** -- it persists a preference value that no code path reads; only Gemini is actually wired.

Future Improvement Areas

(Recommendations -- none of the following exist in the codebase today.)

- Hash stored passwords (e.g. BCrypt) instead of storing them in plaintext, and stop naming the plaintext column `passwordHash`.
- Add a refresh-token-aware re-auth path for Swiggy if/when their MCP API adds one, to avoid the 5-day full-OAuth repeat.
- Instamart (groceries) and Dineout (reservations) integrations were explicitly discussed and scoped **out** of this project by product decision -- Swiggy's MCP platform exposes both, so they remain a plausible future expansion rather than a technical blocker.
- Camera/gallery-based food logging is new in this project and was verified primarily on an emulator with a synthetic camera feed; a fuller pass on a range of real devices (varied camera hardware, permission-flow edge cases) would be worth doing before treating it as production-hardened.
- Wire the Claude option in the AI-engine selector to a real second provider, or remove the selector if it's staying cosmetic.
- Consider migrating off `fallbackToDestructiveMigration()` toward real Room migrations once the schema is closer to stable, so future updates don't silently wipe user data.

Glossary

- **ReAct** -- "Reasoning + Acting"; an agent pattern that interleaves reasoning steps with tool calls. MacroMate's agent is a scripted variant: the tool-calling sequence (search -> fetch menu -> score) is fixed and deterministic, with two narrow points (query interpretation, one bounded re-search) where an LLM is consulted.
- **MCP (Model Context Protocol)** -- an open protocol for exposing tools (like `search_restaurants`) to AI agents/clients over a standard JSON-RPC-based transport. Swiggy's Builders Club platform implements it.
- **PKCE (Proof Key for Code Exchange, RFC 7636)** -- an OAuth 2.1 extension that lets a native app do a secure authorization-code flow without a client secret, by proving possession of a locally-generated random verifier.
- **TDEE (Total Daily Energy Expenditure)** -- the estimated total calories a person burns per day; MacroMate approximates it as `weight (kg) × 30` and adjusts from there for Cut/Bulk/Maintain.
- **Macro(nutrient)** -- protein, carbohydrate, and fat; the three calorie-yielding nutrient categories the app tracks alongside total calories.
- **Variable font** -- a single font file that encodes a continuous range of weights (and sometimes other axes), selected at render time rather than shipping a separate file per weight. MacroMate bundles Outfit and Manrope this way.

Quick Start for Developers

1. **Prerequisites**: Android Studio (or the command-line Gradle/Android SDK toolchain), a JDK (the project has been built here using Android Studio's bundled JBR), an Android emulator or physical device for manual verification.
2. **Clone/open** the project at its root (`macromate/`, containing `settings.gradle.kts`).
3. **Configure the Gemini key (optional)**: `cp .env.example .env`, then set `GEMINI_API_KEY=<your real key>`. Leaving it as the placeholder is fully supported -- the app runs on deterministic fallbacks.
4. **Build**: `./gradlew :app:assembleDebug`
5. **Run unit tests**: `./gradlew :app:testDebugUnitTest`
6. **Install to a running emulator/device**: `adb install -r app/build/outputs/apk/debug/app-debug.apk`
7. **Instrumented tests** (require a connected device/emulator): `./gradlew :app:connectedAndroidTest`
8. **Access the app**: launch "MacroMate" from the device's app drawer, or `adb shell am start -n com.aistudio.macromate.rshqy/com.example.MainActivity`.
9. To exercise the real Swiggy integration, sign up, complete onboarding, then go to Profile -> "Connect Swiggy Account" and complete the phone/OTP login in the browser that opens (requires a real Swiggy account and a device that can actually receive the OTP -- an emulator can open the OAuth page but cannot complete a real phone verification).